

LAPORAN PRAKTIKUM

TUGAS 3 Docker Network, Volume, Bind Mount, tmpfs, dan Compose

Dosen Pengampu : Ferry Astika Saputra



Disusun Oleh :

Adam Rasyid Nurmuhammad

3126640025

D4-IT-B

**PROGRAM STUDI SARJANA TERAPAN
JURUSAN TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026**

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	3
BAB II LANDASAN TEORI	4
BAB III Pengerjaan Praktikum	5
3.1 Persiapan Direktori Kerja dan Status Docker	5
3.2 User-Defined Bridge Network dan DNS Service Discovery	6
3.3 Volume Backup and Restore	6
3.4 Compose multi-container Nginx-Flask-PostgreSQL.....	7
3.5 Analisis Wajib dan Troubleshooting	14
BAB IV EVALUASI DAN LATIHAN MANDIRI	16
BAB V KESIMPULAN.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam lanskap rekayasa perangkat lunak modern dan metodologi DevSecOps, penerapan arsitektur modular seperti multi-container mikroservis telah menjadi standar de-facto. Sistem production umumnya mengisolasi fungsi web server/reverse proxy, aplikasi backend, dan basis data ke dalam container independen. Konfigurasi ini menuntut pemahaman mendalam mengenai segregasi jaringan, penjaminan persistensi data, serta orkestrasi deklaratif siklus hidup kontainer. Docker menyediakan instrumen krusial melalui user-defined bridge network untuk komunikasi antarlayanan yang terisolasi dengan DNS otomatis, serta opsi storage mount (Named Volume, Bind Mount, tmpfs) yang disesuaikan dengan kebutuhan persistensi dan keamanan data. Selain itu, Docker Compose menyatukan orkestrasi multi-container secara reproduktif dan terkelola.

1.2 Tujuan

1. Mampu membuat user-defined bridge network dan membuktikan name resolution otomatis antar container.
2. Membedakan karakteristik, persistensi, portabilitas, dan keamanan antara volume, bind mount, dan tmpfs.
3. Mampu melakukan proses pencadangan (backup) volume data container secara terisolasi menggunakan helper container.
4. Menyusun deklarasi Docker Compose multi-tier (Nginx-Flask-PostgreSQL) dengan pembagian network frontend dan backend.
5. Menerapkan mekanisme service healthcheck dan dependency condition untuk menjamin kesiapan runtime.
6. Mendiagnosis kendala build aplikasi serta merumuskan analisis risiko keamanan dan rekomendasi production

BAB II

LANDASAN TEORI

Docker network berfungsi sebagai boundary arsitektural yang membentuk graf keterjangkauan (reachability graph) antar komponen. Driver bridge menghubungkan container pada satu host menggunakan virtual network interfaces. Berbeda dengan default bridge bawaan Docker, user-defined bridge network menyediakan isolasi yang ketat dan fitur automatic DNS service discovery, di mana container dapat saling berkomunikasi melalui nama kontainer atau nama service alih-alih IP dinamis yang rentan berubah. Port publishing (-p) hanya boleh diterapkan pada service ingress publik, sedangkan komponen sensitif seperti database tetap internal di backend network guna memenuhi prinsip least exposure

BAB III

Pengerjaan Praktikum

3.1 Persiapan Direktori Kerja dan Status Docker

1. Pembuatan Direktori Kerja Bab 3

Membuat direktori kerja terisolasi ~/docker-lab/bab-3 menggunakan perintah mkdir -p dan masuk ke direktori tersebut

```
adam@Devsecops:~$ mkdir -p ~/docker-lab/bab-3
adam@Devsecops:~$ cd ~/docker-lab/bab-3
adam@Devsecops:~/docker-lab/bab-3$ pwd
/home/adam/docker-lab/bab-3
adam@Devsecops:~/docker-lab/bab-3$ ls -la
total 8
drwxrwxr-x 2 adam adam 4096 Sep 12 09:12 .
drwxrwxr-x 5 adam adam 4096 Sep 12 09:12 ..
```

2. Verifikasi Kesiapan Docker Engine

Memeriksa status daemon dan integritas engine melalui docker version dan docker ps sebelum memulai praktikum

```
adam@Devsecops:~/docker-lab/bab-3$ docker version
Client: Docker Engine - Community
Version: 29.8.0
API version: 1.56
Go version: go1.26.8
Git commit: 88096ef
Built: Thu Sep 3 21:50:45 2026
OS/Arch: linux/amd64
Context: default

Server: Docker Engine - Community
Engine:
Version: 29.8.0
API version: 1.56 (minimum version 1.40)
Go version: go1.26.8
Git commit: 3ce5872
Built: Thu Sep 3 21:50:45 2026
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: v2.3.4
GitCommit: db8809540e1a7a9da5d518876894933ff55692ab
runc:
Version: 1.5.1
GitCommit: v1.5.1-0-g8f2685a4
docker-init:
Version: 0.19.0
GitCommit: de40ad0

adam@Devsecops:~/docker-lab/bab-3$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
```

3.2 User-Defined Bridge Network dan DNS Service Discovery

Membuat bridge network lab-net (subnet 172.20.0.0/16) dan menjalankan dua container Nginx (server-a dan server-b). Perintah docker exec server-a ping -c 3 server-b membuktikan DNS service discovery berhasil menerjemahkan nama container secara dinamis

```
adam@Devsecops:~/docker-lab/bab-3$ docker network ls
NETWORK ID      NAME      DRIVER      SCOPE
041949e2b787    bridge    bridge      local
65ce1b444d2c    host      host        local
b0b29c665d76    lab-net   bridge      local
4784c62e83cc    none      null        local
adam@Devsecops:~/docker-lab/bab-3$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS      NAMES
4c756c257e4a   nginx:alpine  "/docker-entrypoint..." About a minute ago Up About a minute 80/tcp     server-b
77beeda4fee8   nginx:alpine  "/docker-entrypoint..." About a minute ago Up About a minute 80/tcp     server-a
adam@Devsecops:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.217 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.105 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.143 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.105/0.155/0.217 ms
```

3.3 Volume Backup and Restore

1. Inisiasi Named Volume 'data-vol'

Membuat volume data-vol dan menjalankan container writer untuk menguji penulisan log waktu secara kontinu ke dalam volume.

```
adam@Devsecops:~/docker-lab/bab-3$ docker volume ls
DRIVER      VOLUME NAME
local       data-vol
adam@Devsecops:~/docker-lab/bab-3$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Sat Sep 12 02:24:38 UTC 2026
Sat Sep 12 02:24:43 UTC 2026
Sat Sep 12 02:24:48 UTC 2026
Sat Sep 12 02:24:53 UTC 2026
Sat Sep 12 02:24:58 UTC 2026
Sat Sep 12 02:25:03 UTC 2026
Sat Sep 12 02:25:08 UTC 2026
Sat Sep 12 02:25:14 UTC 2026
Sat Sep 12 02:25:19 UTC 2026
```

2. Pembuktian Persistensi Data Named Volume

Setelah container writer dihapus paksa (docker rm -f writer), container baru dijalankan me-mount volume yang sama. Pembacaan file log.txt membuktikan data tetap persisten melampaui lifecycle container

```
adam@Devsecops:~/docker-lab/bab-3$ docker rm -f writer
writer
adam@Devsecops:~/docker-lab/bab-3$ docker volume ls
DRIVER      VOLUME NAME
local       data-vol
adam@Devsecops:~/docker-lab/bab-3$ docker run --rm \
-v data-vol:/data \
alpine:3.20 \
cat /data/log.txt
Sat Sep 12 02:24:38 UTC 2026
Sat Sep 12 02:24:43 UTC 2026
Sat Sep 12 02:24:48 UTC 2026
Sat Sep 12 02:24:53 UTC 2026
Sat Sep 12 02:24:58 UTC 2026
Sat Sep 12 02:25:03 UTC 2026
Sat Sep 12 02:25:08 UTC 2026
Sat Sep 12 02:25:14 UTC 2026
Sat Sep 12 02:25:19 UTC 2026
Sat Sep 12 02:25:24 UTC 2026
Sat Sep 12 02:25:29 UTC 2026
Sat Sep 12 02:25:34 UTC 2026
Sat Sep 12 02:25:39 UTC 2026
Sat Sep 12 02:25:44 UTC 2026
Sat Sep 12 02:25:49 UTC 2026
Sat Sep 12 02:25:54 UTC 2026
Sat Sep 12 02:25:59 UTC 2026
Sat Sep 12 02:26:04 UTC 2026
Sat Sep 12 02:26:09 UTC 2026
Sat Sep 12 02:26:14 UTC 2026
Sat Sep 12 02:26:19 UTC 2026
```

3. Backup Named Volume Menggunakan Helper Container

Menjalankan container sementara berbasis Alpine untuk mengompres direktori volume ke dalam file arsip tar: data-vol-backup.tar.gz pada sistem host.

```
adam@Devsecops:~/docker-lab/bab-3$ ls -lh backup/
total 4.0K
-rw-r--r-- 1 root root 217 Sep 12 09:29 data-vol-backup.tar.gz
```

4. Pengujian Bind Mount dengan Sinkronisasi Host

Memetakan direktori host ./html ke dalam container Nginx. Perubahan file HTML di host langsung ter-update secara real-time pada respon server tanpa rebuild image

```
adam@Devsecops:~/docker-lab/bab-3$ curl http://localhost:8080
<h1>Bab 3 DevSecOps</h1>
<p>File berubah dari host.</p>
```

5. Pengujian Karakteristik tmpfs (RAM-Based Storage)

Menguji penyimpanan memori non-persisten menggunakan tmpfs. Data di direktori tmpfs terbukti terhapus secara otomatis begitu container di-restart.

```
adam@Devsecops:~/docker-lab/bab-3$ docker exec tmpfs-test cat /tmp/test.txt
data sementara
adam@Devsecops:~/docker-lab/bab-3$ docker rm -f tmpfs-test
tmpfs-test
adam@Devsecops:~/docker-lab/bab-3$ docker run --rm \
  --tmpfs /tmp \
  alpine:3.20 \
  ls -la /tmp
total 4
drwxrwxrwt  2 root    root      40 Sep 12 02:39 .
drwxr-xr-x  1 root    root     4096 Sep 12 02:39 ..
```

3.4 Compose multi-container Nginx-Flask-PostgreSQL

Arsitektur stack: web (Nginx) berada hanya di network frontend, app (Flask) menghubungkan frontend dan backend, sedangkan db (PostgreSQL) hanya di backend sehingga database tidak terekspos ke internet. Data DB disimpan pada volume pg-data dan Compose menunggu db healthy (pg_isready) sebelum app dijalankan.

1. Membuat compose.yaml

Tulis file compose.yaml sebagai definisi deklaratif stack, berisi 3 service (web = Nginx, app = Flask, db = PostgreSQL), 2 network (frontend, backend), dan 1 named volume (pg-data) beserta healthcheck pada service db.

```
nano compose.yaml
```

```
services:
  web:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - frontend
    depends_on:
      - app

  app:
    build:
      context: ./app
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: labdb
      DB_USER: labuser
      DB_PASS: labpass123
    networks:
      - frontend
      - backend
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
```

2. Membuat Aplikasi Flask

Buat app/requirements.txt:

```
GNU nano 6.2
Flask==3.1.2
psycopg[binary]==3.2.9
unicorn==23.0.0
```

Buat app/app.py:

```
GNU nano 6.2 app/app.py
import os

import psycopg
from flask import Flask, jsonify

app = Flask(__name__)

def database_connection():
    return psycopg.connect(
        host=os.environ["DB_HOST"],
        port=os.getenv("DB_PORT", "5432"),
        dbname=os.environ["DB_NAME"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASS"],
    )

@app.get("/")
def index():
    try:
        with database_connection() as connection:
            with connection.cursor() as cursor:
                cursor.execute("SELECT version();")
                database_version = cursor.fetchone()[0]

        return jsonify(
            status="ok",
            message="Nginx, Flask, dan PostgreSQL berhasil terhubung",
            database=database_version,
        )
    except Exception as error:
        return jsonify(
            status="error",
```

3. Membuat Dockerfile aplikasi

```
GNU nano 6.2 app/Dockerfile *
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

RUN useradd --system --uid 10001 appuser
USER appuser

EXPOSE 5000

CMD ["unicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

4. Membuat konfigurasi Nginx

nano nginx.conf

```
GNU nano 6.2
upstream flask_backend {
    server app:5000;
}

server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://flask_backend;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
    }

    location = /static.html {
        root /usr/share/nginx/html;
    }
}
```

5. Membuat halaman statis opsional

nano html/index.html

```
GNU nano 6.2                                     html/index.html *
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Laboratorium Docker Compose</title>
</head>
<body>
    <h1>Docker Compose Bab 3</h1>
    <p>Bind mount Nginx berhasil digunakan.</p>
</body>
</html>
```

6. Memvalidasi Compose

docker compose config menampilkan model komposisi akhir (hasil interpolation dan merge) sekaligus memvalidasi kebenaran YAML; --services hanya mendaftarkan nama-nama service yang terdefinisi.

docker compose config

```
adam@Devsecops:~/docker-lab/bab-3$ docker compose config
name: bab-3
services:
  app:
    build:
      context: /home/adam/docker-lab/bab-3/app
      dockerfile: Dockerfile
    depends_on:
      db:
        condition: service_healthy
        required: true
    environment:
      DB_HOST: db
      DB_NAME: labdb
      DB_PASS: labpass123
      DB_PORT: "5432"
      DB_USER: labuser
    networks:
      backend: null
      frontend: null
  db:
    environment:
      POSTGRES_DB: labdb
      POSTGRES_PASSWORD: labpass123
      POSTGRES_USER: labuser
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -U labuser -d labdb
      timeout: 5s
      interval: 5s
      retries: 5
```

docker compose config --services

```
adam@Devsecops:~/docker-lab/bab-3$ docker compose config --services
db
app
web
```

docker compose ps

```
adam@Devsecops:~/docker-lab/bab-3$ docker compose ps
NAME                IMAGE              COMMAND                SERVICE  CREATED      STATUS      PORTS
bab-3-app-1         bab-3-app          "gunicorn --bind 0.0.0..."  app      2 minutes ago  Up 2 minutes  5000/tcp
bab-3-db-1          postgres:16-alpine "docker-entrypoint.s..."  db       11 days ago  Up 2 minutes (healthy)  5432/tcp
bab-3-web-1         nginx:alpine       "/docker-entrypoint..."  web      2 minutes ago  Up 2 minutes  127.0.0.1:8080->80/tcp
```

7. Menjalankan Stack

docker compose up -d --build membangun image dan menjalankan semua container di background sesuai model Compose; docker compose ps digunakan untuk mengecek status dan port tiap service.

```
docker compose up -d --build
```

```
adam@Devsecops:~/docker-lab/bab-3$ docker compose up -d --build
[+] Building 41.6s (13/13) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 498B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 307B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/python:3.12-slim@sha256:2f17fc04
=> => resolve docker.io/library/python:3.12-slim@sha256:2f17fc04
=> => sha256:06ad939ed42b51caafb25b14810875d14e0ee0c441d3e2e5e4d
=> => sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db0d
=> => sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240ea35a5ea96a
=> => sha256:b0dc7f87bef15c2536cb182d6c48b52f90a0522884ec9eb222f
=> => extracting sha256:6b37362b3da78869050b894b799ad4df04f1f3b5
=> => extracting sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240
=> => extracting sha256:b0dc7f87bef15c2536cb182d6c48b52f90a0522f
=> => extracting sha256:06ad939ed42b51caafb25b14810875d14e0ee0c4
=> [internal] load build context
=> => transferring context: 1.40kB
=> [2/6] WORKDIR /app
=> [3/6] COPY requirements.txt .
=> [4/6] RUN pip install --no-cache-dir -r requirements.txt
=> [5/6] COPY app.py .
=> [6/6] RUN useradd --system --uid 10001 appuser
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:b299cd6e95fa702dd68ff299d97bd53d
=> => exporting config sha256:83da67715d4aac38b481c408a29c8c041d
=> => exporting attestation manifest sha256:07926ffbf48eaedb361d
=> => exporting manifest list sha256:675e158690f8e81ee987a9fbfe1
=> => naming to docker.io/library/bab-3-app:latest
=> => unpacking to docker.io/library/bab-3-app:latest
=> resolving provenance for metadata file
[+] up 4/4
✔ Image bab-3-app Built
✔ Container bab-3-app-1 Started
✔ Container bab-3-web-1 Started
✔ Container bab-3-db-1 Healthy
```

```
docker compose ps
```

```
adam@Devsecops:~/docker-lab/bab-3$ docker compose ps
NAME                IMAGE              COMMAND                SERVICE    CREATED        STATUS               PORTS
bab-3-app-1         bab-3-app         "gunicorn --bind 0.0..." app         2 minutes ago  Up 2 minutes              5000/tcp
bab-3-db-1          postgres:16-alpine "docker-entrypoint.s..." db         11 days ago   Up 2 minutes (healthy)  5432/tcp
bab-3-web-1         nginx:alpine      "/docker-entrypoint..." web         2 minutes ago  Up 2 minutes              127.0.0.1:8080->80/tcp
```

8. Menguji Aplikasi

```
curl http://localhost:8080/
```

```
curl -i http://localhost:8080/health
```

```
curl http://localhost:8080/index.html
```

```
adam@Devsecops:~/docker-lab/bab-3$ curl http://localhost:8080/
{"database":"PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0"}
adam@Devsecops:~/docker-lab/bab-3$ curl -i http://localhost:8080/health
HTTP/1.1 200 OK
Server: nginx/1.31.5
Date: Wed, 23 Sep 2026 10:51:18 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive

{"status":"healthy"}
adam@Devsecops:~/docker-lab/bab-3$ curl http://localhost:8080/index.html
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Laboratorium Docker Compose</title>
</head>
<body>
  <h1>Docker Compose Bab 3</h1>
  <p>Bind mount Nginx berhasil digunakan.</p>
</body>
</html>
```

9. Memeriksa log

```
docker compose logs --tail 100
```

```
adam@Devsecops:~/docker-lab/bab-3$ docker compose logs --tail 100
web-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will
web-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-en
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-liste
web-1 | 10-listen-on-ipv6-by-default.sh: info: can not modify /etc/ngi
web-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsu
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-
web-1 | /docker-entrypoint.sh: Configuration complete; ready for start
web-1 | 2026/09/23 10:34:32 [notice] 1#1: using the "epoll" event meth
web-1 | 2026/09/23 10:34:32 [notice] 1#1: nginx/1.31.5
web-1 | 2026/09/23 10:34:32 [notice] 1#1: built by gcc 15.2.0 (Alpine
web-1 | 2026/09/23 10:34:32 [notice] 1#1: OS: Linux 6.8.0-138-generic
```

Periksa log service tertentu

```
docker compose logs app
```

```
docker compose logs db
```

```
docker compose logs web
```

```
adam@Devsecops:~/docker-lab/bab-3$ docker compose logs app
docker compose logs db
docker compose logs web
app-1 | [2026-09-23 10:34:32 +0000] [1] [INFO] Starting gunicorn 23
app-1 | [2026-09-23 10:34:32 +0000] [1] [INFO] Listening at: http://
app-1 | [2026-09-23 10:34:32 +0000] [1] [INFO] Using worker: sync
app-1 | [2026-09-23 10:34:32 +0000] [7] [INFO] Booting worker with
app-1 | [2026-09-23 10:34:32 +0000] [8] [INFO] Booting worker with
```

10. Menghentikan Lingkungan

docker compose down menghentikan dan menghapus container serta network proyek; down -v bersifat destruktif karena sekaligus menghapus named volume pg-data (data PostgreSQL ikut hilang).

```
docker compose down
```

```
docker compose down -v -- menghapus container,
network, sekaligus volume PostgreSQL
```

11. Hasil Akhir

```
adam@Devsecops:~/docker-lab$ tree
.
├── bab-2
├── bab-3
│   ├── app
│   │   ├── app.py
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── compose.yaml
│   ├── html
│   │   └── index.html
│   └── nginx.conf
```

3.5 Analisis Wajib dan Troubleshooting

- Masalah: Saat mengakses curl <http://localhost:8080/static.html>, halaman tidak muncul (respon bukan halaman yang diharapkan). Nginx hanya melayani file yang benar-benar ada di direktori html/ yang di-bind mount, sedangkan static.html belum dibuat di folder tersebut.
- Diagnosis: Dengan curl -i <http://localhost:8080/static.html> terlihat kode status respon dari Nginx (404/error), dan pemeriksaan pada direktori html/ menunjukkan bahwa file

static.html memang tidak ada, sehingga Nginx tidak dapat menemukan file yang diminta untuk disajikan.

- Solusi: Membuat file static.html di dalam folder html/. Karena direktori tersebut di-bind mount (./html:/usr/share/nginx/html:ro), file langsung dapat diakses tanpa rebuild image, dan curl <http://localhost:8080/static.html> kemudian berhasil menampilkan halaman tersebut.

BAB IV

EVALUASI DAN LATIHAN MANDIRI

1. Mengapa user-defined bridge lebih baik daripada default bridge untuk multi-container app?

User-defined bridge menyediakan resolusi DNS otomatis berdasarkan nama kontainer/service, menyediakan isolasi jaringan tersegregasi secara default tanpa koneksi liar antar kontainer independen, serta memungkinkan penyesuaian parameter subnet dan gateway secara fleksibel.

2. Apa risiko bind mount terhadap keamanan host?

Bind mount menghubungkan sistem berkas container langsung ke direktori host. Jika container disusupi atau berjalan sebagai root tanpa opsi read-only (:ro), penyerang dapat memodifikasi, menimpa file konfigurasi sistem operasi host, atau menyisipkan skrip berbahaya.

3. Apa perbedaan docker compose down dan docker compose down -v?

docker compose down hanya menghentikan dan menghapus kontainer serta jaringan internal stack, sementara named volume data tetap utuh. Sebaliknya, docker compose down -v secara destruktif turut menghapus named volume beserta seluruh data persisten di dalamnya.

4. Kapan depends_on dengan healthcheck lebih tepat daripada depends_on biasa?

Digunakan ketika service dependen (seperti backend Flask) membutuhkan service prasyarat (seperti PostgreSQL) dalam kondisi siap menerima koneksi (ready state). depends_on standar hanya menunggu proses container dijalankan, bukan kesiapan layanannya.

5. Bagaimana strategi backup volume untuk database produksi?

Strategi backup produksi meliputi: (a) Melakukan logical dump transaksional menggunakan pg_dump secara berkala; (b) Snapshot storage volume secara konsisten; (c) Enkripsi dan replikasi file backup ke storage sekunder/cloud terpisah; (d) Menjalankan uji pemulihan (restore drill) secara rutin

BAB V KESIMPULAN

Praktikum Bab 3 telah berhasil mengonseptualisasikan dan mengimplementasikan pilar fundamental jaringan, persistensi data, dan orkestrasi multi-container pada Docker. Penggunaan user-defined bridge membuktikan name resolution otomatis mampu menghadirkan service discovery dinamis yang tangguh. Pemisahan mekanisme storage (Volume, Bind Mount, dan tmpfs) berhasil diterapkan sesuai karakteristik beban kerja data. Implementasi Docker Compose dengan pembagian network tier (frontend dan backend) serta integrasi healthcheck memastikan ketergantungan antar komponen tersinkronisasi dengan stabil dan aman. Penerapan analisis risiko keamanan dan rekomendasi arsitektural menjadi bekal penting dalam merancang pipeline DevSecOps yang aman, andal, dan siap produksi